

Введение

Компьютерное зрение является областью искусственного интеллекта, в которой изображения анализируются программными методами. Однако перед тем как алгоритм сможет выделять объекты, распознавать текст или выполнять классификацию, изображение необходимо подготовить. Поэтому предобработка является одним из базовых этапов любой системы компьютерного зрения.

Предобработка позволяет уменьшить шум, привести изображение к удобному цветовому представлению, повысить контраст отдельных элементов, выделить границы и подготовить бинарные маски. В данной работе рассматриваются основные методы обработки изображений средствами библиотеки OpenCV.

Отчёт оформлен по логике разработки программы: сначала формулируется задача, затем приводятся теоретические сведения, после этого описываются этапы реализации, структура финального кода, результаты экспериментов и итоговые выводы.

1 Цель и постановка задачи

Цель работы — изучить основные методы предобработки изображений и реализовать программный инструмент для наглядного сравнения их влияния на разные типы изображений.

В рамках работы необходимо выполнить следующие задачи:

- подготовить не менее трёх тестовых изображений: портрет, городской пейзаж и изображение с текстом;
- загрузить изображения с помощью `cv2.imread()` и корректно вывести их через `matplotlib`;
- учесть различие между порядком каналов BGR в OpenCV и RGB в `matplotlib`;
- преобразовать изображения в RGB, Grayscale и HSV;
- вывести отдельные каналы HSV и объяснить их назначение;
- исследовать работу `GaussianBlur`, `medianBlur` и `blur` с размерами ядра 5×5 , 11×11 и 21×21 ;

- применить фильтр повышения резкости на основе ядра свёртки;
- получить бинарное изображение и применить морфологические операции;
- добавить гауссов шум и шум «соль-перец», после чего сравнить способы их удаления;
- собрать итоговую визуализацию в виде сравнительной сетки и сохранить её в PNG.

2 Используемые инструменты

Для выполнения работы используется язык программирования Python и набор библиотек, предназначенных для обработки изображений, математических операций и визуализации результатов.

Таблица 1 — Используемые инструменты

Инструмент	Назначение	Роль в работе
Python	Язык программирования	Реализация всей логики эксперимента
OpenCV	Библиотека компьютерного зрения	Загрузка, преобразование и фильтрация изображений
NumPy	Работа с массивами и матрицами	Создание ядер свёртки и программное добавление шума
Matplotlib	Визуализация данных	Построение сеток изображений и сохранение PNG
Jupyter Notebook / Python script	Среда выполнения	Пошаговый запуск и проверка результатов

3 Теоретические сведения

3.1 Цветовые пространства RGB, Grayscale и HSV

OpenCV по умолчанию читает изображения в порядке каналов BGR. Matplotlib ожидает порядок RGB, поэтому перед отображением необходимо выполнить преобразование `cv2.cvtColor(img, cv2.COLOR_BGR2RGB)`. Без этого красный и синий каналы меняются местами, и цвета выглядят неестественно.

RGB описывает цвет через интенсивность красного, зелёного и синего каналов. Такое представление удобно для отображения изображения на экране, но не всегда удобно для выделения объектов по цвету, потому что цвет и освещённость смешаны между собой.

HSV разделяет цвет на оттенок, насыщенность и яркость. Канал H отвечает за сам цвет, канал S показывает насыщенность, а канал V отвечает за яркость. Поэтому при выделении объекта по цвету HSV удобнее RGB: цветовая информация сосредоточена в одном канале, а яркость отделена от оттенка.

3.2 Сглаживающие фильтры

Сглаживающие фильтры применяются для уменьшения шума и подавления мелких деталей. Они работают с окрестностью пикселя, которая задаётся размером ядра. Чем больше ядро, тем сильнее размытие и тем больше деталей теряется.

В работе используются три фильтра: гауссово размытие, медианный фильтр и простое усреднение. Гауссово размытие учитывает вес пикселей по гауссовому распределению, медианный фильтр заменяет пиксель медианным значением из окрестности, а простое усреднение заменяет пиксель средним значением соседей.

3.3 Повышение резкости

Повышение резкости реализуется с помощью операции свёртки. В работе используется ядро 3×3 , в котором центральный пиксель получает больший вес, а соседние пиксели имеют отрицательные коэффициенты. За счёт этого усиливаются локальные перепады яркости, то есть границы и текстуры становятся более выраженными.

Недостатком повышения резкости является усиление шума. Поэтому резкость и чистота изображения находятся в конфликте: метод делает изображение визуально чётче, но одновременно подчёркивает случайные артефакты.

3.4 Морфологические операции

Морфологические операции применяются к бинарным изображениям и маскам. Они изменяют форму белых объектов относительно выбранного структурного элемента. В работе используются эрозия, дилатация, открытие и закрытие.

Эрозия уменьшает белые области, дилатация расширяет их. Открытие представляет собой эрозию с последующей дилатацией и применяется для удаления мелких белых шумов. Закрытие состоит из дилатации и последующей эрозии и используется для заполнения мелких чёрных разрывов внутри объектов.

3.5 Исследование шума

В работе рассматриваются два типа шума: гауссов шум и шум «соль-перец». Гауссов шум представляет собой случайное отклонение яркости пикселей по нормальному распределению. Для его подавления хорошо подходит GaussianBlur.

Шум «соль-перец» проявляется в виде случайных чёрных и белых пикселей. Для его удаления эффективен медианный фильтр, потому что одиночные выбросы оказываются крайними значениями и не попадают в медиану.

4 Проектирование программы

4.1 Общая структура проекта

Финальная версия программы реализована с разделением на классы. Такой подход выбран для того, чтобы отделить загрузку данных, обработку изображений, генерацию шумов, морфологические операции и визуализацию результатов.

Общая структура проекта имеет следующий вид:

```
project/
├── main.py
├── images/
│   ├── portrait.jpg
│   ├── city.jpg
│   └── text.jpg
```

4.2 Классы финальной версии программы

Таблица 2 — Классы финальной версии программы

Класс	Назначение
ImageItem	Хранит имя изображения, путь к файлу и массив пикселей в формате BGR.
ImageRepository	Загружает изображения из папки images и проверяет наличие файлов.
ColorSpaceProcessor	Выполняет преобразования RGB, Grayscale, HSV и выделяет каналы HSV.
FilterProcessor	Содержит методы GaussianBlur, medianBlur, blur и фильтр повышения резкости.
NoiseProcessor	Добавляет гауссов шум и шум «соль-перец».
MorphologyProcessor	Выполняет бинаризацию, эрозию, дилатацию, opening и closing.
Visualizer	Создаёт сетки изображений, выводит результаты и сохраняет PNG-файлы.
PreprocessingExperiment	Управляет всей последовательностью эксперимента.

5 Ход выполнения работы

5.1 Загрузка и отображение изображений

На первом этапе были подготовлены три изображения разных типов: портрет человека, городской пейзаж и изображение с текстом. Изображения загружаются с помощью `cv2.imread()`, после чего перед выводом переводятся из BGR в RGB.

Место для вставки изображения: outputs/01_originals.png

Рисунок 1 — Исходные изображения

5.2 Цветовые пространства

Для каждого изображения были получены представления RGB, Grayscale и HSV. Отдельно выведены каналы H, S и V. Это позволяет увидеть, что информация об оттенке в HSV отделена от яркости, поэтому такое пространство удобно при фильтрации объектов по цвету.

Место для вставки изображения: outputs/02_color_spaces_portrait.png

Рисунок 2 — Цветовые пространства для портрета

Место для вставки изображения: outputs/02_color_spaces_city.png

Рисунок 3 — Цветовые пространства для городского пейзажа

Место для вставки изображения: outputs/02_color_spaces_text.png

Рисунок 4 — Цветовые пространства для изображения с текстом

5.3 Сглаживающие фильтры

На выбранном изображении были применены три сглаживающих фильтра: GaussianBlur, medianBlur и blur. Для каждого фильтра использованы размеры ядра 5×5 , 11×11 и 21×21 . В результате получена сетка 3×3 , по которой можно сравнить силу размытия и степень потери деталей.

Место для вставки изображения: outputs/03_smoothing_filters.png

Рисунок 5 — Сравнение сглаживающих фильтров

5.4 Фильтр резкости

Для повышения резкости использовано ядро свёртки 3×3 с центральным коэффициентом 5 и четырьмя соседними коэффициентами -1. После

применения фильтра границы и текстуры становятся более заметными, но также усиливается шум.

Место для вставки изображения: `outputs/04_sharpening.png`

Рисунок 6 — Изображение до и после повышения резкости

5.5 Морфологические операции

Для выполнения морфологических операций изображение с текстом было переведено в оттенки серого и бинаризовано методом Оцу. После этого были применены эрозия, дилатация, открытие и закрытие со структурными элементами 3×3 , 5×5 и 9×9 .

Место для вставки изображения: `outputs/05_morphology.png`

Рисунок 7 — Морфологические операции над бинарным изображением

5.6 Исследование шума

К изображению программно добавлены два типа шума: гауссов шум и шум «соль-перец». После этого к каждому зашумлённому изображению применены `GaussianBlur`, `medianBlur` и `blur`. Сравнение показывает, что гауссов шум лучше подавляется `GaussianBlur`, а шум «соль-перец» — медианным фильтром.

Место для вставки изображения: `outputs/06_gaussian_noise_reduction.png`

Рисунок 8 — Подавление гауссова шума

Место для вставки изображения: `outputs/06_salt_pepper_noise_reduction.png`

Рисунок 9 — Подавление шума «соль-перец»

5.7 Итоговая визуализация

В финале программа формирует единую сравнительную сетку. Строками выступают три исходных изображения, а столбцами — методы обработки: исходник, `Grayscale`, `GaussianBlur`, `MedianBlur`, резкость и морфология. Полученная сетка сохраняется как отдельный PNG-файл для вставки в отчёт.

Место для вставки изображения: `outputs/07_final_comparison.png`

Рисунок 10 — Итоговая сравнительная таблица методов предобработки

6 Сравнительный анализ методов

Таблица 4 — Сравнительный анализ методов предобработки

Метод	Что делает	Когда использовать	Ограничения
RGB	Отображает цвет через красный, зелёный и синий каналы.	Для обычного вывода изображения.	Неудобен для устойчивого выделения цвета при разном освещении.
Grayscale	Убирает цвет и оставляет яркость.	Для бинаризации, фильтрации, поиска контуров и текста.	Теряется информация о цвете.
HSV	Разделяет оттенки, насыщенность и яркость.	Для выделения объектов по цвету.	Требуется подбирать диапазоны H, S и V.
GaussianBlur	Размывает изображение с весами по гауссовому распределению.	Для подавления гауссова шума.	Размывает границы и мелкие детали.
MedianBlur	Заменяет пиксель медианным значением из окрестности.	Для удаления шума «соль-перец».	Может исказить тонкие линии при большом ядре.
AverageBlur	Заменяет пиксель средним значением соседей.	Для простого сглаживания.	Сильнее съедает детали и работает посредственно с выбросами.
Sharpening	Усиливает локальные	Для подчёркивания	Одновременно

	перепады яркости.	границ и текстур.	усиливает шум.
Morphology	Изменяет форму объектов на бинарной маске.	Для очистки масок, удаления мелкого шума и заполнения разрывов.	Работает только после бинаризации или получения маски.

По результатам сравнения можно сделать следующие выводы:

- HSV удобнее RGB для выделения объектов по цвету, так как яркость отделена от оттенка;
- GaussianBlur лучше подходит для подавления гауссова шума;
- medianBlur наиболее эффективен против шума «соль-перец»;
- простое усреднение работает универсально, но часто сильнее портит детали;
- фильтр резкости повышает визуальную чёткость, но усиливает шум;
- морфологические операции эффективны только после получения бинарной маски.

Заключение

В ходе работы была разработана программа для исследования основных методов предобработки изображений. Были реализованы загрузка и отображение изображений, преобразование цветовых пространств, сглаживающие фильтры, повышение резкости, бинаризация, морфологические операции и исследование двух типов шума.

Практическое сравнение показало, что фильтр необходимо выбирать под конкретную задачу. Для цветовой сегментации удобнее использовать HSV, для гауссова шума — GaussianBlur, для шума «соль-перец» — medianBlur. Морфологические операции применяются не к обычным цветным изображениям, а к бинарным маскам, поэтому они используются как инструмент очистки результата сегментации.

Главный результат работы состоит в понимании того, что фильтрация изображения является математической операцией над пикселем и его окрестностью. Это не универсальная

«магическая» обработка, а набор инструментов, каждый из которых имеет собственное назначение и ограничения.

Приложение А. Листинг финального кода

Ниже приведён финальный вариант файла `main.py`, реализованный с разделением на классы.

```
import os
import cv2
import numpy as np
import matplotlib.pyplot as plt

from dataclasses import dataclass

@dataclass
class ImageItem:
    name: str
    path: str
    bgr: np.ndarray

class ImageRepository:
    def __init__(self, image_paths):
        self.image_paths = image_paths

    def load(self):
        images = []

        for name, path in self.image_paths.items():
            image = cv2.imread(path)

            if image is None:
                raise FileNotFoundError(f"Image not found: {path}")

            images.append(ImageItem(name=name, path=path, bgr=image))

        return images

class ColorSpaceProcessor:
    def to_rgb(self, image):
        return cv2.cvtColor(image, cv2.COLOR_BGR2RGB)

    def to_grayscale(self, image):
        return cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

    def to_hsv(self, image):
        return cv2.cvtColor(image, cv2.COLOR_BGR2HSV)

    def get_hsv_channels(self, image):
        hsv = self.to_hsv(image)
        return cv2.split(hsv)

class FilterProcessor:
    def gaussian_blur(self, image, kernel_size):
        return cv2.GaussianBlur(image, (kernel_size, kernel_size), 0)
```

```

def median_blur(self, image, kernel_size):
    return cv2.medianBlur(image, kernel_size)

def average_blur(self, image, kernel_size):
    return cv2.blur(image, (kernel_size, kernel_size))

def sharpen(self, image):
    kernel = np.array([
        [0, -1, 0],
        [-1, 5, -1],
        [0, -1, 0]
    ])

    return cv2.filter2D(image, -1, kernel)

def get_smoothing_results(self, image, kernel_sizes):
    results = []

    for size in kernel_sizes:
        results.append((f"Gaussian {size}x{size}", self.gaussian_blur(image, size)))
        results.append((f"Median {size}x{size}", self.median_blur(image, size)))
        results.append((f"Average {size}x{size}", self.average_blur(image, size)))

    return results

class NoiseProcessor:
    def add_gaussian_noise(self, image, mean=0, sigma=25):
        noise = np.random.normal(mean, sigma, image.shape)
        noisy = image.astype(np.float32) + noise
        return np.clip(noisy, 0, 255).astype(np.uint8)

    def add_salt_pepper_noise(self, image, amount=0.03):
        noisy = image.copy()
        total_pixels = image.shape[0] * image.shape[1]

        salt_count = int(total_pixels * amount / 2)
        pepper_count = int(total_pixels * amount / 2)

        salt_y = np.random.randint(0, image.shape[0], salt_count)
        salt_x = np.random.randint(0, image.shape[1], salt_count)

        pepper_y = np.random.randint(0, image.shape[0], pepper_count)
        pepper_x = np.random.randint(0, image.shape[1], pepper_count)

        noisy[salt_y, salt_x] = 255
        noisy[pepper_y, pepper_x] = 0

    return noisy

class MorphologyProcessor:
    def __init__(self, color_processor):
        self.color_processor = color_processor

    def binarize(self, image):
        gray = self.color_processor.to_grayscale(image)
        _, binary = cv2.threshold(gray, 0, 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU)

```

```

        return binary

    def erode(self, binary, kernel_size):
        kernel = np.ones((kernel_size, kernel_size), np.uint8)
        return cv2.erode(binary, kernel, iterations=1)

    def dilate(self, binary, kernel_size):
        kernel = np.ones((kernel_size, kernel_size), np.uint8)
        return cv2.dilate(binary, kernel, iterations=1)

    def opening(self, binary, kernel_size):
        kernel = np.ones((kernel_size, kernel_size), np.uint8)
        return cv2.morphologyEx(binary, cv2.MORPH_OPEN, kernel)

    def closing(self, binary, kernel_size):
        kernel = np.ones((kernel_size, kernel_size), np.uint8)
        return cv2.morphologyEx(binary, cv2.MORPH_CLOSE, kernel)

    def get_morphology_results(self, image, kernel_sizes):
        binary = self.binarize(image)
        results = [("Binary", binary)]

        for size in kernel_sizes:
            results.append((f"Erosion {size}x{size}", self.erode(binary, size)))
            results.append((f"Dilation {size}x{size}", self.dilate(binary, size)))
            results.append((f"Opening {size}x{size}", self.opening(binary, size)))
            results.append((f"Closing {size}x{size}", self.closing(binary, size)))

        return results

class Visualizer:
    def __init__(self, color_processor, output_dir="outputs"):
        self.color_processor = color_processor
        self.output_dir = output_dir
        os.makedirs(output_dir, exist_ok=True)

    def prepare_image(self, image):
        if len(image.shape) == 2:
            return image, "gray"

        return self.color_processor.to_rgb(image), None

    def show_and_save_grid(self, title, items, rows, cols, filename, figsize=(16, 10)):
        plt.figure(figsize=figsize)
        plt.suptitle(title, fontsize=16)

        for index, (item_title, image) in enumerate(items, start=1):
            prepared_image, cmap = self.prepare_image(image)

            plt.subplot(rows, cols, index)
            plt.imshow(prepared_image, cmap=cmap)
            plt.title(item_title)
            plt.axis("off")

        plt.tight_layout()

        output_path = os.path.join(self.output_dir, filename)
        plt.savefig(output_path, dpi=200, bbox_inches="tight")
        plt.show()

```

```

def show_originals(self, images):
    items = [(item.name, item.bgr) for item in images]
    self.show_and_save_grid(
        title="Original images",
        items=items,
        rows=1,
        cols=len(items),
        filename="01_originals.png",
        figsize=(15, 5)
    )

def show_color_spaces(self, image_item):
    rgb = self.color_processor.to_rgb(image_item.bgr)
    grayscale = self.color_processor.to_grayscale(image_item.bgr)
    h, s, v = self.color_processor.get_hsv_channels(image_item.bgr)

    items = [
        ("RGB", cv2.cvtColor(rgb, cv2.COLOR_RGB2BGR)),
        ("Grayscale", grayscale),
        ("HSV Hue", h),
        ("HSV Saturation", s),
        ("HSV Value", v)
    ]

    self.show_and_save_grid(
        title=f"Color spaces: {image_item.name}",
        items=items,
        rows=1,
        cols=5,
        filename=f"02_color_spaces_{image_item.name.lower()}.png",
        figsize=(18, 4)
    )

def show_smoothing(self, image_item, smoothing_results):
    self.show_and_save_grid(
        title=f"Smoothing filters: {image_item.name}",
        items=smoothing_results,
        rows=3,
        cols=3,
        filename="03_smoothing_filters.png",
        figsize=(15, 12)
    )

def show_sharpening(self, image_item, sharpened):
    items = [
        ("Before", image_item.bgr),
        ("After", sharpened)
    ]

    self.show_and_save_grid(
        title=f"Sharpening: {image_item.name}",
        items=items,
        rows=1,
        cols=2,
        filename="04_sharpening.png",
        figsize=(12, 5)
    )

def show_morphology(self, image_item, morphology_results):
    self.show_and_save_grid(
        title=f"Morphology: {image_item.name}",
        items=morphology_results,
        rows=4,
        cols=4,
        filename="05_morphology.png",
    )

```

```

        figsize=(18, 16)
    )

def show_noise_reduction(self, title, original, noisy, filtered_results, filename):
    items = [
        ("Original", original),
        ("Noisy", noisy)
    ]

    items.extend(filtered_results)

    self.show_and_save_grid(
        title=title,
        items=items,
        rows=1,
        cols=5,
        filename=filename,
        figsize=(18, 4)
    )

def show_final_comparison(self, images, filter_processor, morphology_processor):
    rows = len(images)
    cols = 6

    plt.figure(figsize=(18, 5 * rows))
    plt.suptitle("Final preprocessing comparison", fontsize=16)

    for row_index, image_item in enumerate(images):
        original = image_item.bgr
        grayscale = self.color_processor.to_grayscale(original)
        gaussian = filter_processor.gaussian_blur(original, 11)
        median = filter_processor.median_blur(original, 11)
        sharpened = filter_processor.sharpen(original)
        binary = morphology_processor.binarize(original)
        morph = morphology_processor.opening(binary, 5)

        row_items = [
            ("Original", original),
            ("Grayscale", grayscale),
            ("GaussianBlur", gaussian),
            ("MedianBlur", median),
            ("Sharpening", sharpened),
            ("Morphology", morph)
        ]

        for col_index, (title, image) in enumerate(row_items):
            prepared_image, cmap = self.prepare_image(image)
            plot_index = row_index * cols + col_index + 1

            plt.subplot(rows, cols, plot_index)
            plt.imshow(prepared_image, cmap=cmap)
            plt.title(f"{image_item.name}\n{title}")
            plt.axis("off")

    plt.tight_layout()

    output_path = os.path.join(self.output_dir, "07_final_comparison.png")
    plt.savefig(output_path, dpi=200, bbox_inches="tight")
    plt.show()

class PreprocessingExperiment:
    def __init__(self, image_paths):

```

```

self.image_paths = image_paths
self.repository = ImageRepository(image_paths)
self.color_processor = ColorSpaceProcessor()
self.filter_processor = FilterProcessor()
self.noise_processor = NoiseProcessor()
self.morphology_processor = MorphologyProcessor(self.color_processor)
self.visualizer = Visualizer(self.color_processor)

def run(self):
    images = self.repository.load()

    self.visualizer.show_originals(images)

    for image_item in images:
        self.visualizer.show_color_spaces(image_item)

    selected_image = images[0]
    text_image = images[2]

    smoothing_results = self.filter_processor.get_smoothing_results(
        selected_image.bgr,
        [5, 11, 21]
    )

    self.visualizer.show_smoothing(selected_image, smoothing_results)

    sharpened = self.filter_processor.sharpen(selected_image.bgr)
    self.visualizer.show_sharpening(selected_image, sharpened)

    morphology_results = self.morphology_processor.get_morphology_results(
        text_image.bgr,
        [3, 5, 9]
    )

    self.visualizer.show_morphology(text_image, morphology_results)

    gaussian_noisy = self.noise_processor.add_gaussian_noise(selected_image.bgr)
    salt_pepper_noisy = self.noise_processor.add_salt_pepper_noise(selected_image.bgr)

    gaussian_filtered = [
        ("GaussianBlur", self.filter_processor.gaussian_blur(gaussian_noisy, 5)),
        ("MedianBlur", self.filter_processor.median_blur(gaussian_noisy, 5)),
        ("AverageBlur", self.filter_processor.average_blur(gaussian_noisy, 5))
    ]

    salt_pepper_filtered = [
        ("GaussianBlur", self.filter_processor.gaussian_blur(salt_pepper_noisy, 5)),
        ("MedianBlur", self.filter_processor.median_blur(salt_pepper_noisy, 5)),
        ("AverageBlur", self.filter_processor.average_blur(salt_pepper_noisy, 5))
    ]

    self.visualizer.show_noise_reduction(
        title="Gaussian noise reduction",
        original=selected_image.bgr,
        noisy=gaussian_noisy,
        filtered_results=gaussian_filtered,
        filename="06_gaussian_noise_reduction.png"
    )

    self.visualizer.show_noise_reduction(
        title="Salt and pepper noise reduction",
        original=selected_image.bgr,

```

```

        noisy=salt_pepper_noisy,
        filtered_results=salt_pepper_filtered,
        filename="06_salt_pepper_noise_reduction.png"
    )

    self.visualizer.show_final_comparison(
        images=images,
        filter_processor=self.filter_processor,
        morphology_processor=self.morphology_processor
    )

def main():
    image_paths = {
        "Portrait": "images/portrait.jpg",
        "City": "images/city.jpg",
        "Text": "images/text.jpg"
    }

    experiment = PreprocessingExperiment(image_paths)
    experiment.run()

if __name__ == "__main__":
    main()

```